UCUG1903 Signals and Information Technology

2026-27 Fall Term, Assignment 1

In this assignment, you will learn how to use MATLAB to create a sinusoidal signal and perform various operations on it. Please submit your MATLAB scripts along with a Word or PDF report that includes screenshots and brief explanations on Canvas by September 9, 2026, at 23:59. Late submissions are accepted but will incur a 50% deduction per day.

Task 1: Generate a Sinusoidal Signal [50]

a) Problem description

A signal represents the variation of a physical quantity over one or more independent variables. In this task, you are required to write a MATLAB function that generates a sinusoidal signal, where the quantity varies with time.

Let y denote the amplitude of the quantity. A continuous-time sinusoidal signal is mathematically defined as:

$$y(t) = Asin(2\pi ft)$$

where:

- A is the amplitude of the signal,
- f is the frequency in Hertz (Hz),
- t is the continuous time variable.

When t takes continuous values, the above equation describes a continuous-time signal.

In MATLAB, as in most engineering applications, it is impossible to store or process a truly continuous signal directly. Instead, we use a sequence of discrete sample points to approximate the signal. As long as the sampling interval (step size Δt) is sufficiently small, the discrete representation will visually match its continuous counterpart.

Assuming we focus on a 1-second segment of the sinusoidal signal, if we use f_s equally spaced samples to represent this segment, then the time spacing (i.e., the interval) between consecutive samples is $\Delta t = 1/f_s$. This quantity f_s is called the *sampling rate* and is measured in samples per second (Hz).

For a total duration of T_{duration} seconds, the total number of samples N is given by:

$$N = f_s \times T_{\text{duration}}$$

Let the integer $n = 0,1,2,\dots,N-1$ denote the sample index. The discrete-time signal is then expressed as:

$$x[n] = x(n\Delta t) = A\sin(2\pi fn\Delta t),\ n = 0,1,\dots,N-1$$

Important Note: The last point of the time vector should be $T_{\text{duration}} - \Delta t$, not T_{duration} itself. Since we start at $t = 0$, the N-th point lands exactly at $(N-1)\Delta t$. This ensures that the signal length exactly matches the specified duration and avoids repeating the endpoint of the period when plotting.

b) Implementation instructions

Open the file generate_sinusoid.m in your project. The function interface (inputs and outputs) has already been defined for you. Complete the following two core steps inside the function:

Step 1: Create the time base.

Generate a column vector t based on the given sample_rate and duration.

Hint: Calculate the total number of samples N first. Use the colon operator : to generate an index sequence from 0 to $N-1$,

then multiply by Δt (or use linspace). Remember to transpose the result to obtain a column vector.

Step 2: Generate the sinusoid.

Using the given freq (frequency) and amplitude (amplitude), compute the signal vector y from the time vector t created in Step 1.

Hint: MATLAB's built-in sin function is vectorized—it can accept a vector as input and compute the sine for every element in a single operation, without requiring a for loop.

You can verify your generated signal using the provided file plot_sinusoid.m. In your report, please include screenshots of the signals generated with different parameter settings, and briefly explain what you observe from each waveform.

In MATLAB, a function is defined using the function keyword. The general syntax is:

```
function [output1, output2, ...] = functionName(input1, input2, ...)
```

- The function body contains the executable code that performs the required computation.
- The function must be saved in a .m file whose name matches the function name (e.g., generate_sinusoid.m).
- Variables defined inside the function are local to its workspace, meaning they do not interfere with variables in the base workspace unless explicitly passed as inputs or outputs.
- The help text (the comment block immediately following the function line) is displayed when the user types help functionName in the command window, making it an essential part of code documentation.

In MATLAB, the sin function is defined as following.

Syntax

```
Y = sin(X)
```

Description

Y = sin(X) returns the sine of the elements of X. The sin function operates element-wise on arrays. The function accepts both real and complex inputs.

- For real values of X, sin(X) returns real values in the interval [-1, 1].
- For complex values of X, sin(X) returns complex values.

Task2: Operation on Signals [50]

a) Problem description

In signal processing, we often need to modify signals to study system responses, extract features, or simulate real-world scenarios. The fundamental operations on discrete-time signals include:

Operation	Mathematical Description	Effect
Time Shift	$y_{\text{shift}}[n] = y[n - n_0]$	**Moves the signal forward (delay) or backward (advance) in time**
Amplitude Scaling	$y_{\text{scaled}}[n] = a \cdot y[n]$	**Changes the magnitude of the signal**
Time Scaling	$y_{\text{scaled}}[n] = y[n/a]$	**Compresses (a > 1) or expands (a < 1) the signal in time**
Time Inversion	$y_{\text{inv}}[n] = y[-n]$	**Flips the signal with respect to the time origin**
Addition	$y_{\text{sum}}[n] = y_1[n] + y_2[n]$	**Combines two signals by point-wise addition**
Multiplication	$y_{\text{prod}}[n] = y_1[n] \cdot y_2[n]$	**Combines two signals by point-wise multiplication**

In this task, you will apply these operations to the sinusoidal signal you generated in Task 1. This will help you understand how each operation changes the signal's appearance in the time domain

b) Implementation Guide

- **Step 1: Implement Time Shift**

Open the file time_shift.m and implement the time shift operation. The input shift_time is given in seconds : a positive value delays the signal (shifts it to the right on the time axis), while a negative value advances it (shifts it to the left).

Hint: Time shifting does not change the shape of the signal; only its position on the time axis changes

- **Step 2: Implement Amplitude Scaling**

Open the file amplitude_scale.m and implement the amplitude scaling operation. The input scale_factor is a multiplier: a value greater than 1 increases the signal amplitude, between 0 and 1 decreases it, and a negative value both scales and inverts the signal.

Hint: Amplitude scaling applies the same multiplier to every value in the signal vector. The time vector remains unchang

- **Step 3: Implement Time Scaling**

Open the file time_scale.m and implement the time scaling operation. The input scale_factor compresses (scale_factor > 1) or expands (0 < scale_factor < 1) the signal in time, which changes its frequency.

Hint: be sure that scale_factor in this time is bigger than 0. This work assume the same number of samples before and after the scaling, thus the time step between the samples could be changed.

- **Step 4: Implement Time Inversion**

Open the file time_invert.m and implement the time inversion operation. Time inversion flips the signal with respect to the time origin ($t = 0$).

Hint: This operation reverses the order of the data sequence, rather than simply negating the amplitude values. Consider how to reorder the elements of a vector and its corresponding time instants.

- **Step 5: Implement Signal Addition**

Open the file signal_add.m and implement the signal addition operation. This adds two signals point-by-point.
Hint: Both input signals are assumed to have the same length and correspond to the same time instants.

- **Step 6: Implement Signal Multiplication**

Open the file signal_multiply.m and implement the signal multiplication operation. This multiplies two signals point-by-point.
Hint: Both input signals are assumed to have the same length and correspond to the same time instants.

flipud

Flip array up to down

collapse all in page

Syntax

B = flipud(A)

Description

B = flipud(A) returns A with its rows flipped in the up-down direction (that is, about a horizontal axis).

If A is a column vector, then flipud(A) returns a vector of the same length with the order of its elements reversed. If A is a row vector, then flipud(A) simply returns A. For multidimensional arrays, flipud operates on the planes formed by the first and second dimensions.

Example:

A = (1:10)'

A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]^T

Use flipud to flip the elements of A in the vertical direction.

B = flipud(A)

B = [10, 9, 8,7, 6, 5, 4, 3, 2, 1]^T

The order of the elements in B is reversed compared to A.